

This document describes the second project in the course, devoted to two time integration methods for second-order ODEs, i.e., Newmark- β and HHT- α . This assignment has 5 tasks.

Task 1

Study the Newmark- β method and the HHT- α method from the lecture notes. Distinguish three cases:

- The implicit method with damping: $C \neq 0$
- The implicit method without damping: $C = 0$
- The explicit method: $C = 0$, $\beta = 0$ and $\gamma = 1/2$

Task 2

Many second-order problems, for example the pendulum in Project 1, can be rewritten as a system of first-order ODEs so that standard ODE solvers can be used. If a second-order problem is already written in such a first-order formulation, but one wants to use Newmark- β or HHT- α methods to directly solve the original second-order system, the transformation needs to be undone.

Design a new problem class `Explicit_Problem_2nd` in `Assimulo`, derived from the existing `Explicit_Problem` class, which works as a wrapper class and can essentially undo this transformation.

Hint: In this task, we only deal with first-order systems that we already knew were transformed from second-order systems, and that the first half of the first-order ODEs were used to define the auxiliary ‘velocity’ variables, while the second half were from the dynamics, the latter of which is what you need to extract and reformulate into your `Explicit_Problem_2nd`.

Task 3

Implement the Newmark- β method and the HHT- α method in `Assimulo`. To do so, derive a class `2nd_Order` from `Explicit_ODE`, which should serve as a base class for the two newly implemented methods.

Task 4

Solve the elastic pendulum problem in Project 1 using the explicit version of the Newmark- β method. Test your code by comparing its results with those by the time integration methods in Project 1.

Task 5

Download and run the elastodynamic problem `elastodyn.py`. Replace the Asimulo solvers with your own implementations of the Newmark- β and HHT- α methods. Note: This may require writing a new `rhs` or extracting the mass and stiffness matrices from the `elastodynamic_beam` class:

```
Beam = elastodynamic_beam(dx, T=tend)
M = Beam.Mass_mat
K = Beam.Stiffness_mat
C = Beam.Damping_mat # Remark: You need to set eta_M and eta_K
f = Beam.F
```

Examine the function `rhs` to see how to stop the forcing $f(t)$.